

Index

Presentation av verktyg: .NET, Azure, GitHub, Docker.....	2
0. Inledning	2
1. Översikt av verktygsval och motivering till valen.....	2
2. .NET och Blazor Server (mjukvarukod, backend och webfront)	2
2.1 .NET.....	3
2.2 Blazor Server	3
3. Azure och CosmosDB (molntjänster och databas)	3
3.1 Azure Service Bus.....	4
3.2 Azure Functions	4
3.3 Azure Application Insights.....	4
3.4 Azure App Service.....	5
3.5 Azure App Configuration	5
3.6 Azure Key Vault och roll-baserad åtkomst (Managed Identity + RBAC).	5
3.7 Azure Storage Account.....	5
3.8 CosmosDB serverless	5
4. GitHub, GHCR och Docker (CI/CD och containerisering)	5
4.1 GitHub och GitHub Actions.....	6
4.3 Docker och containrar	6
4.4 Bicep (Infrastructure as Code, förkortat IaC)	6
5. Testning, kvalitetssäkring och ytterligare verktyg.....	7
5.1 xUnit och NSubstitute.....	7
5.2 QRCode.....	7
6. Huvudfunktionen - Den Stora Röda Knappen	7
7. Sammanfattning och samspel.....	8
7.1 Synkront system:.....	8
7.2 Eventdrivet system:	8
7.3 Hur Azure-tjänsterna samverkar.....	9
7.4 Överblick och analogi	9
8. Referenser.....	10
8.1 Viktiga ADRer för verktygsval:	10

Presentation av verktyg: .NET, Azure, GitHub, Docker

0. Inledning

Fullständigt repo för projektet kan hittas här:

<https://github.com/mymh13/clo24-nikhal78-examwork>

Projektet har en extensiv dokumentation som återfinns i foldern docs. En hel del är teknisk presentation på engelska, men det finns en del mer lättviktigt material som förklarar syfte och kontext. Följande länkar är mer lättillgängliga och bra för att greppa helheten:

[Glossary](#) – Förklarar begrepp, förkortningar och termer som används i projektet.

[Systemöversikt](#) – Ger en vid överblick av tanken bakom systemet innan det började byggas.

1. Översikt av verktygsval och motivering till valen

Kunden har idag en applikation där ett äldre och ett nyare system fungerar sida vid sida. Vissa verktyg har refaktorerats och integrerats i det nya verktyget, medan andra lever kvar i det äldre. På grund av det så arbetar kunden i dagsläget i två system, samtidigt.

Målet med detta projekt är att skapa ett gränssnitt för att kunna hantera båda systemen inom samma flöde, den tekniska lösningen blir ett verktyg för att successivt kunna flytta över funktionalitet från det gamla till det nya systemet.

För att uppnå detta har jag valt att använda Microsofts .NET-ekosystem tillsammans med Azure-moljtjänster. Detta val motiveras av att kunden redan använder Microsoft-teknologier i stor utsträckning, vilket gör integrationen enklare och mer kostnadseffektiv.

2. .NET och Blazor Server (mjukvarukod, backend och webfront)

När vi besöker en webbplats eller använder en applikation, då möts vi av användargränssnittet, det som kallas "frontend". Det är knapparna vi klickar på, formulären vi fyller i och allt visuellt som presenteras i webbläsaren. I vår projekt byggs detta med Blazor Server.

Logiken som ligger bakom gränssnittet: hur data hanteras, hur systemet reagerar och vilka regler som gäller, det kallas "backend". Den delen bygger vi med ASP.NET.

En fördel i vårt projekt är att både frontend och backend använder samma programmeringsspråk: C# (uttalas "C sharp"). Det gör att de två delarna kan samarbeta på ett enkelt och effektivt sätt.

2.1 .NET

.NET är Microsofts plattform för att bygga applikationer. Tänk på det som en verktygslåda som innehåller allt en behöver för att skapa, köra och underhålla webbapplikationer. I det här projektet använder vi .NET 8, samma version som kunden redan använder. Det ger stabilitet och gör det lättare att integrera med befintliga system.

2.2 Blazor Server

Blazor Server är en del av .NET-ekosystemet som gör det möjligt för oss att bygga interaktiva webbgränssnitt utan att behöva bygga applikationen i ett externt ramverk som exempelvis JavaScript. För användaren innebär det en snabb, responsiv sida, även om den första inladdningen kan ta något längre tid jämfört med traditionella JavaScript-baserade lösningar som exempelvis React eller Vue.

Blazor server har en stor styrka: eftersom frontend och backend delar programmeringspråk och verktyg, då blir systemet enklare att bygga, underhålla och vidareutveckla.

Om vi tänker oss att kollektivtrafiken är .NET och Blazor Server själva fordonen, dvs bussarna och tågen som tar resenärerna från punkt A till punkt B. Fordonen följer en bestämd rutt (applikationslogiken) och transporterar användaren genom olika funktioner i systemet.

3. Azure och CosmosDB (molntjänster och databas)

Azure är Microsofts molnplattform. En plats där vi kan köra våra applikationer utan att behöva äga eller underhålla servrar själva. *En bra liknelse är en stor parkeringsplats där vi hyr utrymme (vid behov, eller över längre perioder) och bara betalar för den del vi använder.*

Projektet använder flera Azure-tjänster som var och en har sitt tydliga syfte. I GitHub-repositoriet presenteras de i en översikt där varje tjänst listas med sitt syfte och vilken kostnadsplan vi använder:

Service	Purpose	Tier/Plan
App Service	Run Blazor Server and API application in the same App Service plan.	Basic B1 (dev)
App Configuration	Centralized management of settings and feature flags between environments. Hot-reload via sentinel key pattern.	Free (dev)
Key Vault	Secure storage of secrets and connection strings. Managed identity access via RBAC.	Standard
Application Insights	Logging, monitoring, and performance measurement via KQL. Used by both App Service and Function App. Custom events track dual-system architecture flows.	Pay-as-you-go
API Management (APIM)	Gateway for public GET endpoints, caching, and rate limiting.	(Future)
Cosmos DB	Serverless NoSQL database for bookings, users, and outbox events. Partition key strategy for efficient querying.	Serverless
Service Bus	Message queue for event-driven architecture. <code>booking-events</code> queue with dead letter queue support.	Basic (dev)
Azure Functions	Serverless event processing. Processes Service Bus messages for event-driven flow.	Basic B1 (dev)
Storage Account	Required for Azure Functions runtime and state management.	Standard LRS

Nedan följer en överiktlig genomgång av de tjänster vi använder:

3.1 Azure Service Bus

Service Bus är ett system för att skicka meddelanden mellan olika delar av applikationen. När någonting händer (till exempel "en biljett har bokats"), skickas ett meddelande som andra komponenter kan reagera på.

Service Bus fungerar som högtalarsystemet på en station. När någonting behöver kommuniceras ropas det ut till rätt aktörer. Om någon inte är redo just då, väntar meddelandet i kön tills mottagaren kan hantera det.

3.2 Azure Functions

Detta är små program som körs automatiskt när de får ett meddelande från Service Bus. De utför en uppgift, till exempel att bearbeta en ny bokning.

Functions är som automatiska dörrar eller sensorer på stationen. När rätt signal kommer aktiveras de direkt och gör sin specifika uppgift.

3.3 Azure Application Insights

Som namnet antyder så ger Insights just insikt och en överblick av din applikation. Både App Service och Functions använder Insights för att visa loggar och realtidsdata. Fel, prestanda, användarstatistik.

I projektet används Insights även för en visualiserad "Workbook" där vi kan se skillnaden mellan synkron och eventdriven drift. Detta ger konkret pedagogiskt värde för demo-syfte.

Insights är stationens kameror och sensorer. De visar hur allting fungerar i realtid och hjälper oss att upptäcka problem.

3.4 Azure App Service

App Service är platsen där vår webbapplikation körs. Det är som en parkeringsplats för hemsidan/appen. Azure sköter drift och underhåll, vilket gör att vi inte behöver hantera egna servrar.

Med andra ord är App Service som själva buss- eller tågstationen, det är dit användarna (resenärerna) kommer för att interagera med systemet.

3.5 Azure App Configuration

Här lagrar vi systeminställningar och så kallade "feature flags" (funktionsflaggor) som aktiverar eller av-aktiverar funktionalitet utan att vi behöver starta om applikationen.

Det är kontrollpanelen på stationen där vi slår av och på olika funktioner, ungefär som belysning, högtalare eller spårväxlar.

3.6 Azure Key Vault och roll-baserad åtkomst (Managed Identity + RBAC).

I Key Vault lagras data som vi vill säkra från åtkomst, lösenord, anslutningsinformation. Endast godkända komponenter i systemet får komma åt den informationen, via säkra identitetsformer.

Vi kan se detta som ett säkert kassaskåp på stationen. Allt värdefullt och känsligt förvaras där, med strikta regler för vem som får öppna kassaskåpet.

3.7 Azure Storage Account

Azure Functions behöver ett Storage Account för att lagra tillstånd och kördata. Det är en teknisk förutsättning, men ingenting som användaren märker av eller interagerar med.

Det är som förrådsutrymmet bakom stationen. Nödvändig för driften och personalen, men ingenting som resenären ser.

3.8 CosmosDB serverless

CosmosDB är vår databas, det är där all information om biljetter, användare och bokningar lagras. Vi använder "serverless"-läget, vilket betyder att vi bara betalar för det vi faktiskt använder, inte för att hålla databasen igång.

CosmosDB är biljettkontoret eller biljettautomaten. Det är där information om bokningar sparas. Serverless innebär att företaget endast betalar när någon köper en biljett, inte när automaten står stilla.

4. GitHub, GHCR och Docker (CI/CD och containerisering)

För att få koden från utvecklarens dator till tjänsterna på molnet så använder projektet tre centrala byggstenar: GitHub, GitHub Container Registry (GHCR) och Docker. Dessa hanterar versionshistorik, automatiska bygg- och leveransprocesser samt paketering av applikationen.

4.1 GitHub och GitHub Actions

GitHub är platsen där all källkod lagras och versionshanteras. Här samarbetar utvecklare och delar ändringar på ett strukturerat sätt. Utöver detta, använder projektet GitHub Actions, ett system som automatiskt bygger och deployar applikationen så fort ny kod laddas upp.

”Bygger och deployar” betyder i sammanhanget att den kör koden till en körbar applikation och placerar koden fysiskt i ett sammanhang/en plats där den kan köras ifrån.

När en utvecklare gör en förändring:

1. Koden skickas till GitHub
2. GitHub Actions startar automatiskt
3. Systemet bygger applikationen, testar den och leverar den till Azure eller till containrar i GHCR

GitHub är som ett centralt arkiv och administrationskontor där all information om systemet förvaras. GitHub Actions är de automatiska arbetarna som ser till att varje ny version byggs, kontrolleras och installeras direkt, utan att vi behöver göra det manuellt.

4.3 Docker och containrar

Verktyget vi använder för att skapa dessa containrar heter Docker. Det paketerar applikationen tillsammans med dess beroenden: rätt versioner av ramverk, bibliotek och konfigurationer. (Moderna programmeringsspråk återanvänder gärna kod-bibliotek som andra utvecklare redan skrivit, för att inte behöva göra jobbet en gång till).

Det här betyder att applikationen beter sig likadant oavsett var vi kör den: exempelvis på utvecklarens dator, på en server, i molnet.

Docker-containrar är som standardiserade transportlådor. Varje låda innehåller exakt det som behövs, och avsett var du placerar den så fungerar innehållet precis likadant. Detta skapar en förutsägbar och stabil driftsmiljö.

4.4 Bicep (Infrastructure as Code, förkortat IaC)

Bicep är det verktyg vi använder för att beskriva hela vår Azure-infrastruktur som kod.

Med Bicep kan vi:

- Skapa och uppdatera Azure-resurser på ett repeterbart sätt
- Versionshantera infrastrukturen precis som vi gör med koden
- Undvika manuella och felbenägna konfigurationer i Azure Portal

Bicep är ritningen för hela stationen. Genom att följa ritningen kan vi bygga samma station igen och igen, med samma resultat varje gång.

5. Testning, kvalitetssäkring och ytterligare verktyg

För att säkerställa att systemet fungerar stabilt, förutsägbart och enligt krav använder projektet flera verktyg för testning och kvalitetssäkring. Dessa verktyg hjälper oss att upptäcka fel tidigt, förbättra tillförlitligheten och skapa en trygg utvecklingsprocess för både kunden och de som förvaltar koden.

5.1 xUnit och NSubstitute

xUnit är ett testramverk för .NET som låter oss skriva automatiserade tester för systemets logik. Tillsammans med NSubstitute, ett verktyg som skapar så kallade "mock-objekt" (simulerade versioner av verkliga komponenter), kan vi testa funktioner utan att koppla in externa system som databaser eller APIer.

Det innebär att vi i trygg miljö kan kontrollera hur systemet beter sig i olika situationer, även sådana som är svåra att återskapa manuellt. Vi mockade exempelvis data i en situation där vi annars hade behövt skapa test-data på den riktiga databasen. Det finns multipla anledningar till att inte skriva testdata på den riktiga databasen, så mockning var en utmärkt lösning här.

xUnit och NSubstitute fungerar som kvalitetskontrollen i kollektivtraffiken. Innan ett tåg eller en buss sätts i trafik genomförs tester för att säkerställa att allting fungerar. På samma sätt testar vi vår kod automatiskt innan nya versioner går ut i drift.

5.2 QRCoder

Detta är ett bibliotek som används för att generera QR-koder i samband med aktivering av biljetter. Den QR-koden kan senare scannas för att verifiera att biljetten är giltig.

QRCoder fungerar som biljettstämpeln. Den skapar en unik markering som gör att biljetten kan kontrolleras och godkännas vid behov.

6. Huvudfunktionen – Den Stora Röda Knappen

Event-Driven Architecture Status:

App Configuration: ✓ Configured (examwork-appconfig-dev) - Sentinel: 1764766992

Feature Manager: ✓ Available - BookingEvents_Enabled = True

Outbox Service: ✓ Registered - Operational (0 pending events)

Toggle Feature Flag (Disable Event-Driven Mode)

Det mest centrala verktyget i lösningen är Toggle-knappen, "Den Stora Röda Knappen". Den styr vilket av två system som hanterar biljettbokningen. Utan att gräva ner sig för djupt i tekniska detaljer så kan vi helt enkelt säga att knappen växlar mellan:

- Ett [synkront system](#), där bokningen går direkt till databasen, och
- Ett [eventdrivet system](#), där bokningen först mellanlagras och därefter bearbetas via Service Bus och Functions

Runt denna funktion finns flera samverkande komponenter som tillsammans möjliggör beteendet. Det handlar om ett antal beroenden: konfiguration, meddelandehantering, databearbetning, identiteter och övervakning. Alla behöver arbeta i symbios för att Toggle-knappen skall fungera som tänkt.

Toggle-knappen är som en växelspak på stationen. Med ett enkelt läge ändrar vi vilket spår bokningen färdas på. Det ena spåret är snabbt och direkt (synkront). Det andra tar en omväg via köer och automatiskt styrda system (eventdrivet).

7. Sammanfattning och samspel

Flödet mellan de olika delarna kan beskrivas visuellt med ett flödesschema. Grundstrukturen följer två möjliga vägar: den snabba synkrona, och den eventdrivna varianten.

Infoga här ett flödesschema eller liknande

7.1 Synkront system:

Flöde:

Användare → Blazor Server → API → Cosmos DB

I det synkrona systemet behandlas bokningen omedelbart. APIet tar emot bokningen, skriver direkt till CosmosDB och svarar sedan tillbaka till användaren. Detta är den enklaste och mest direkta vägen.

Fördelar: Snabb respons, lätt att förstå.

Nackdelar: Mindre robust vid hög belastning, mindre transaktionssäkert.

7.2 Eventdrivet system:

Flöde:

Användare → BlazorS → API → Outbox → Service Bus → Azure Functions → Cosmos DB

I det eventdrivna systemet lagras bokningen först som ett meddelande i en Outbox. Sedan skickas det vidare till Service Bus, som i sin tur triggar en Azure Function som utför den slutliga bearbetningen och uppdaterar Cosmos DB.

Fördelar: Skalar bättre, tål högre belastning, isolerar fel.

Nackdelar: Något längre fördröjning innan bokningen är helt klart, mer komplext system.

7.3 Hur Azure-tjänsterna samverkar

Det eventdrivna systemet bygger på att flera Azure-tjänster arbetar tillsammans:

- App Configuration styr vilket system som används (synkront vs eventdrivet). Toggle-knappen läser denna inställning i realtid.
- Service Bus tar emot och köar händelser från API:et. Detta skapar ett stabilt och buffrat flöde som tål både toppar och störningar.
- Azure Functions aktiveras automatiskt när ett meddelande kommer in i kön. Den ansvarar för den faktiska bearbetningen och uppdateringen av databasen.
- Storage Account lagrar teknisk kördata som Functions behöver (tillstånd, historik, timers).
- Application Insights övervakar helheten och gör det möjligt att följa flödet i realtid. Både när det är synkront och när det är eventdrivet.
- Key Vault och Managed Identity säkerställer att både API, Functions och andra delar har korrekt åtkomst till databasen och övriga resurser – utan att använda lösenord i koden.
- Cosmos DB är slutstationen där bokningen lagras oavsett vilket system som användes.

7.3.1 Samspel i praktiken:

- API:et skriver aldrig direkt till Service Bus – det skriver först till Outbox, vilket garanterar att ingen händelse tappas bort.
- Service Bus garanterar att meddelandet når Functions, även om Functions är tillfälligt upptaget.
- Functions ser till att meddelandet bearbetas exakt en gång. Säkrar data.
- Insights följer hela kedjan och visar tydligt vilken väg ett meddelande färdats.
- App Configuration styr allt detta utan att kräva att applikationen startas om.

Det är denna kombination som gör Toggle-knappen möjlig.

7.4 Överblick och analogi

Båda systemen kan köras parallellt, och vi kan växla mellan dem med Toggle-knappen. Alla Azure-tjänster arbetar tillsammans för att skapa en robust och skalbar lösning, där varje tjänst har sitt specifika syfte och ansvar.

Tänk på det som ett kollektivtrafiksystem: Det finns flera olika sätt att resa (synkront eller eventdrivet). Flera olika stationer och terminaler (Azure-tjänster), och ett centralt kontrollsystem (Toggle-knappen) som styr vilket system som används. Allt används tillsammans för att säkerställa att passagerarna (användarna) kan köpa biljetter och resa smidigt, oavsett vilket system som används.

8. Referenser

Min dokumentation i githubrepo har täckt referensmaterielet mer än väl. ADR-sektionen i repots har verktygsspecifika referenser i slutet av varje ADR. ADR står för Architectural Design Records och betyder helt enkelt ett beslutsunderlag med motivering till varför jag gjort de designval jag gjort. Du hittar dem här:

<https://github.com/mymh13/clo24-nikhal78-examwork/tree/main/docs/adr>

8.1 Viktiga ADRer för verktygsval:

- ADR-003: Bicep för Infrastructure as Code
- ADR-004: Blazor Server för frontend
- ADR-005: Azure-tjänster (App Service, Cosmos DB, Service Bus, Functions, etc.)
- ADR-008: Docker och GitHub Actions för CI/CD